Reviewer Pack — Free Entropy Distinguishes Read-Twice BPs from IP_2 Trivial …

Entry a289b27fd581 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-04-30 17:44:06 UTC

Free Entropy Distinguishes Read-Twice BPs from IP_2 Trivial Ones

Entry ID: `a289b27fd581` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-04-30 17:44:06 UTC

1. Conjecture

Field A (mathematical branch): Free Probability **Field B** (complexity object): Read-Twice Branching Programs

Statement:

> For a read-twice branching program P with size s, the free entropy $\phi(M_P)$ of its transition matrix M_P satisfies $\phi(M_P) = O(\log s)$. For the IP_2 trivial BP, $\phi(M_P) = \Omega(n)$.

Rationale (proposer's reasoning):

> Free entropy quantifies non-commutative randomness; read-twice BPs have structured transitions with low entropy, while IP_2's inherent complexity creates high entropy. This bridges free probability's matrix analysis to BP size lower bounds.

Taxonomy category: `BP_READTWICE` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `7a43751d47eddd8c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	UNCERTAIN	SAFE
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    n = 40
    ε = 1e-3

    def tensor_product(A, B):
        result = []
        for a_row in A:
            new_row = []
            for b_col in zip(*B):
                new_row.append([a * b for a, b in zip(a_row, b_col)])
            result.extend(new_row)
        return result

    def determinant(matrix):
        if len(matrix) == 1:
            return matrix[0][0]
        det = Fraction(0)
        sign = 1
        for j in range(len(matrix)):
            submatrix = [row[:j] + row[j+1:] for row in matrix[1:]]
            det += sign * matrix[0][j] * determinant(submatrix)
            sign *= -1
        return det

    def free_entropy(M):
        det = determinant([[M[i][j] + ε for j in range(len(M))] for i in range(len(M))])
        if det <= 0:
            return float('-inf')
        return (1 / n) * math.log(det)

    random.seed(seed)
    M_P = [[0] * (2 ** n) for _ in range(2 ** n)]
    local_matrices = [random.choice([[Fraction(1, 2), Fraction(1, 2)], [Fraction(1, 2), -Fraction(

    for i in range(n):
        M_P = tensor_product(M_P, local_matrices[i])

    ϕ_M_P = free_entropy(M_P)

    return {
        "metric_name": "free entropy",
```

```
        "metric_value": ϕ_M_P,
        "instances_tested": 1,
        "conjecture_holds": ϕ_M_P <= math.log(n),
        "counterexample": "" if ϕ_M_P <= math.log(n) else "IP_2 trivial BP"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"IP_2 trivial BP\" first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac243985.py", line 71, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ac243985.py", line 49, in run_trial
    M_P = [[0] * (2 ** n) for _ in range(2 ** n)]
          ~~~~^~~~~~~~~~
MemoryError
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed with MemoryError, preventing data collection. Pre-registered support condition cannot be evaluated without successful trials. | next: Optimize matrix storage (e.g., use sparse representations) and retry with smaller n

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	66785
2	preregistration	ollama_remote	qwen3:8b	0	0	24062
3	novelty	ollama_remote	qwen3:8b	0	0	20682
4	novelty	ollama_remote	qwen3:8b	0	0	13999
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12284
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8144
7	judge	ollama_remote	qwen3:8b	0	0	18668

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 164625 ms total latency. Provider mix: {'ollama_remote': 7}

(full prompt+response transcripts available in `research/audit/a289b27fd581.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/a289b27fd581.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/a289b27fd581.tar.gz` (if generated)